hashlock.

Security Audit

Paymatic (Escrow)

May 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

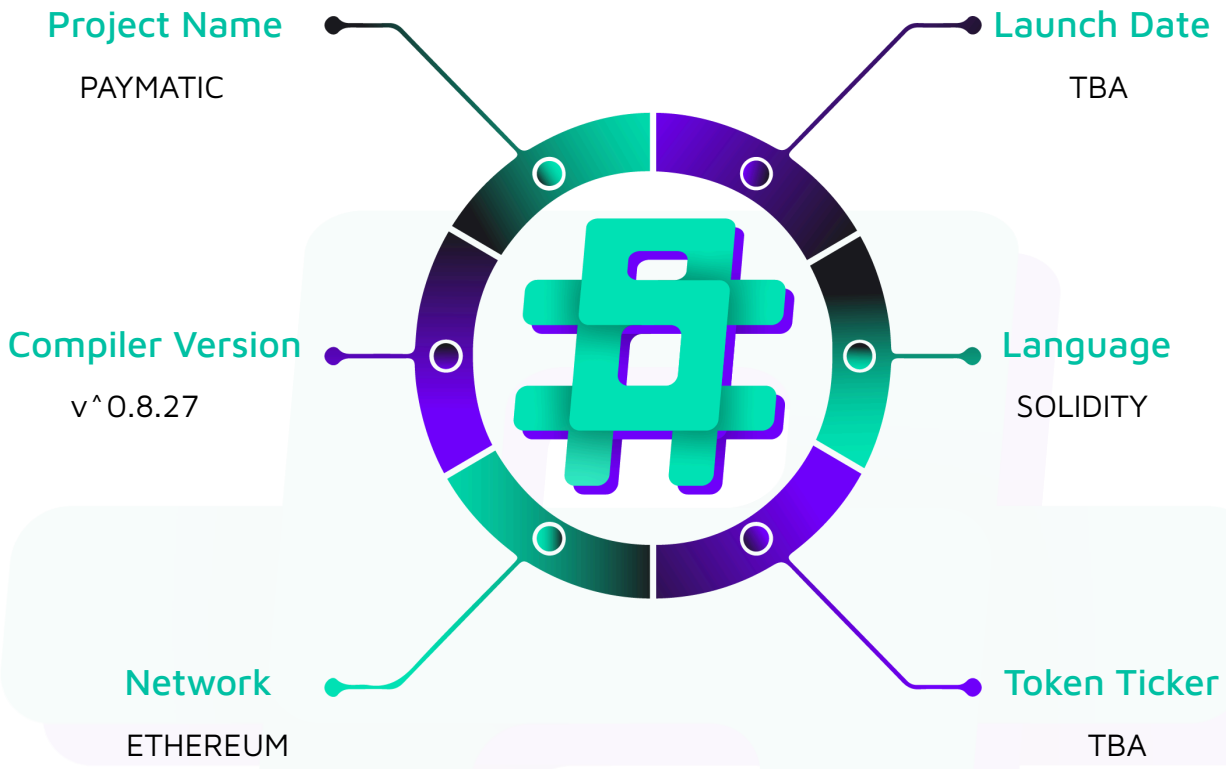
The Paymatic team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

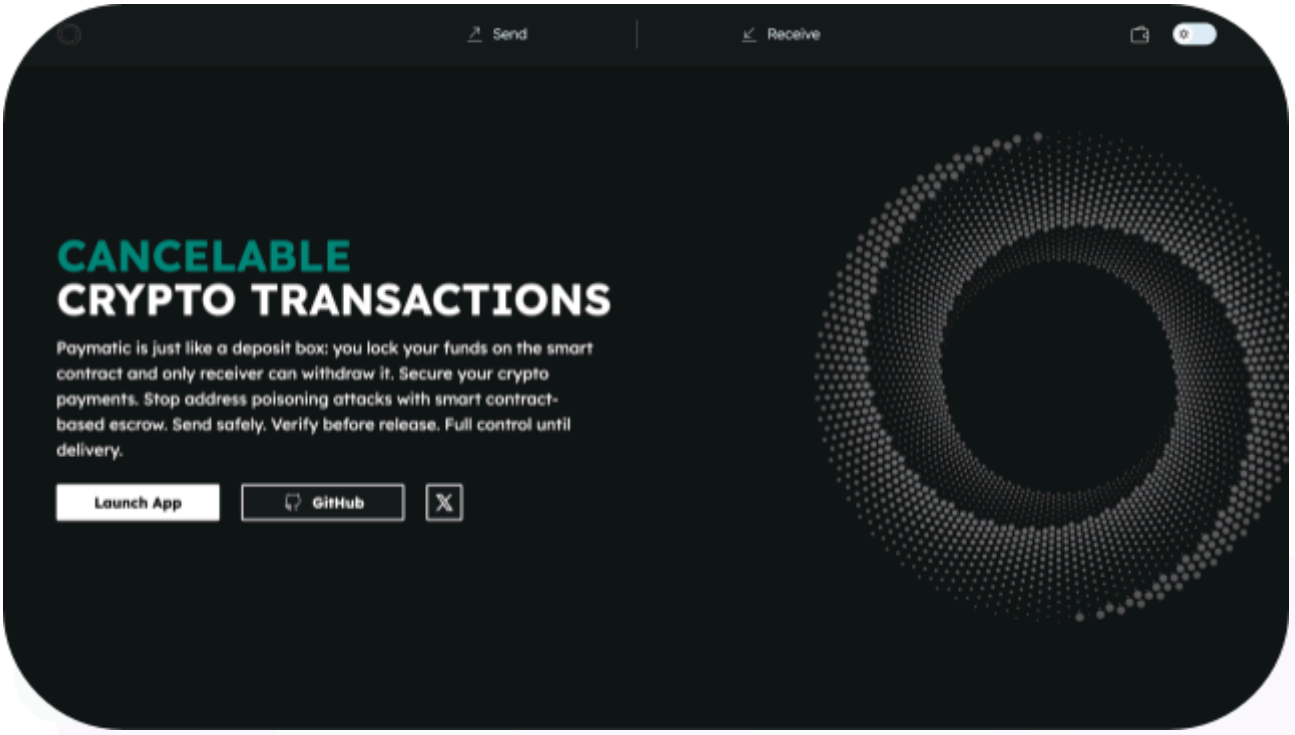
Paymatic is a decentralized escrow platform designed to enhance the security of cryptocurrency transactions. Instead of sending funds directly to a recipient's wallet, users lock their assets in a smart contract. This mechanism ensures that only the intended recipient can withdraw the funds after a predefined grace period, allowing the sender time to verify transaction details and prevent errors. By implementing this approach, Paymatic aims to mitigate risks such as address poisoning attacks and accidental transfers, providing users with greater control and peace of mind in their crypto dealings.

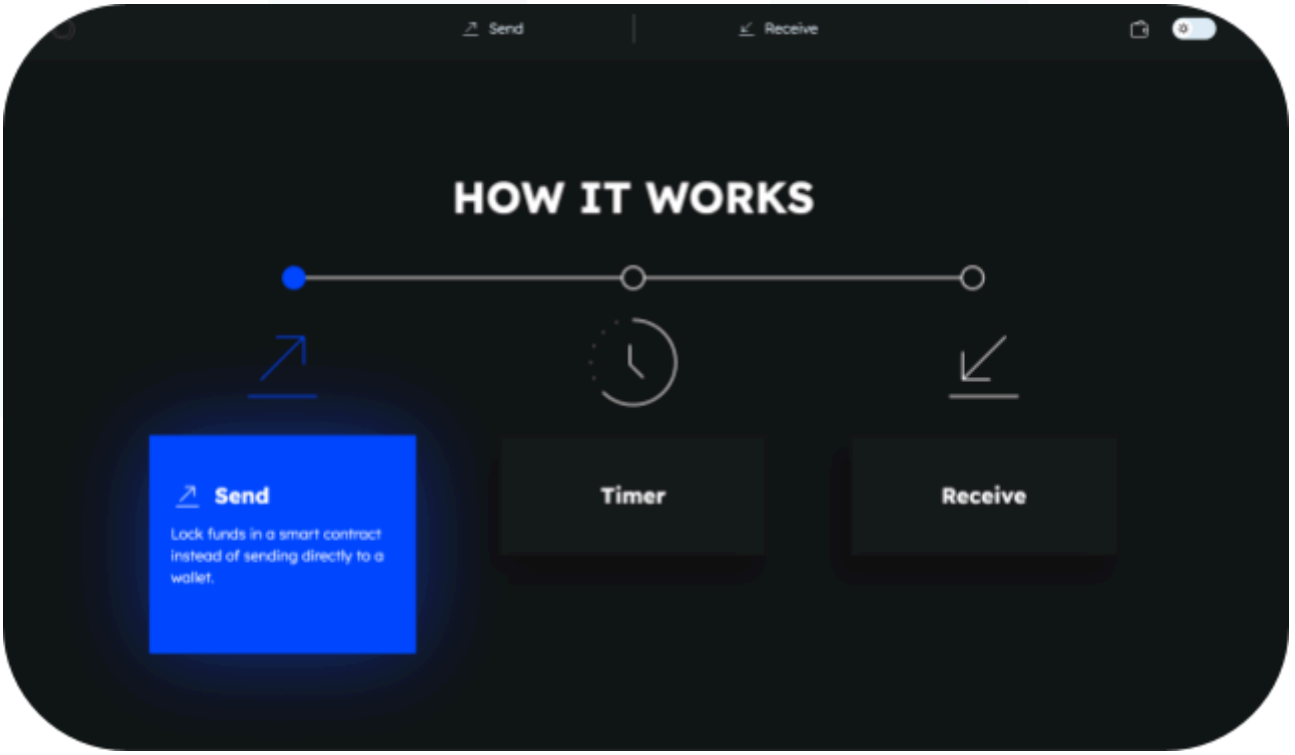
Project Name: Paymatic
Project Type: Escrow
Compiler Version: ^0.8.27
Website: https://paymatic.xyz/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Paymatic project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Paymatic Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**May, 2025**
Contract 1	PaymaticPayments.sol
Contract Address	0x8b918eEB12312e30324712b47C89EC5472394E05
Audited GitHub Commit Hash	95a57a51486eb58e5cf67266ca46e06589adde15
Fix Review GitHub Commit Hash	cca7801fa37755f424de85acb46800dc012cfe55

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

2 Medium severity vulnerabilities

2 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
PaymaticPayments.sol - Allows users to: - CreateERC20Lock - CreateERC20Lock with own timelock period - Cancel the payment before claiming - Settle the payments - Allows admins to: - Pause/UnPause the contracts - Can do an emergencyCancle of a payment - Change Fee Recipient - Change Fee Value - Transfer ownership (from Ownable)	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Paymatic project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Paymatic project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] PaymaticPayments#_createERC20Payment - Potential Loss with Fee-on-Transfer Tokens

Description

The contract does not properly handle Fee-on transfer tokens, which deduct a fee when transferring. It calculates the fee based on the input amount rather than the actual received amount, which will lead to discrepancies. The main issue comes into picture as the protocol is recording the user given 'amount' variable instead of checking the actual amount received.

For example, let's say there is USER A who deposited 100 tokens(Fee-on Transfer type) using the `createERC20Payment` function. Assuming there is a 10% fee, the actual amount of tokens that the PaymaticPayment contract receives is 90. But the contract is recording that the amount it received is 100.
When the receiver tries to redeem the funds, as there are only 90 tokens instead of 100, the transfer fails leading to reverts.

Impact

When using fee-on transfer tokens, the contract will receive less tokens the specified amount but will still record the user input, leading to accounting errors and potential protocol losses when the payment is released.

Recommendation

Calculate the actual received amount after the transfer and use that for payment accounting.

```
// Get balance before transfer
uint256 preBalance = token.balanceOf(address(this));
```

```
// Transfer tokens
SafeERC20.safeTransferFrom(token, msg.sender, address(this), amount);

// Calculate actual received amount
uint256 actualReceived = token.balanceOf(address(this)) - preBalance;

// Calculate fee based on actual received amount
(uint256 feeAmount, uint256 paymentAmount) = _processFee(actualReceived);

// Record the actual amount after fees
payments[idCounter].amount = paymentAmount;
```

Status

Resolved

Medium

[M-01] PaymaticPayments#constructor - Lack of Validation for Owner Address

Description

The constructor accepts an `initialOwner` parameter but does not validate if it is the zero-address or the address the protocol is expecting. Setting the owner to address(0) or to any wrong address would permanently break all owner-restricted functionality in the contract.

Impact

If deployed with the zero address or wrong address as owner, all admin-restricted functions would be permanently inaccessible, including fee management.

Recommendation

Set the `initialOwner` to `msg.sender` and later transfer the admin rights to the expected address.

```
constructor() Ownable(msg.sender) {
    feeRecipient = msg.sender;
    feeValue = 300;

    emit FeeRecipientChanged(initialOwner);
    emit FeeValueChanged(feeValue);
}
```

Status

Acknowledged

[M-02] PaymaticPayments#setFeeValue - Unbounded Fee Value Setting

Description

The `setFeeValue` function allows the owner to set an arbitrary fee value without any upper bound. This could potentially allow setting a fee so high that it consumes the entire payment amount or could even revert due to underflow.

Impact

A malicious or compromised owner could set an extremely high fee (e.g., 1,000,000 basis points or 10,000%), effectively reverting or could set it to 100% to steal the complete user funds as fees.

Recommendation

Implement an upper bound for the fee value, such as 2000 basis points (20% at max).

```
function setFeeValue(uint256 newFeeValue) external onlyOwner {
    require(newFeeValue <= 2000, "Fee cannot exceed 20%");
    feeValue = newFeeValue;
    emit FeeValueChanged(newFeeValue);
}
```

Status

Resolved

Low

[L-01] PaymaticPayments#setFeeReceipent - No Address(0) check for Fee Recipient

Description

The setFeeReceipent function does not validate if the new fee recipient is the zero address or not. Setting the fee recipient to address(0) would cause fees to be permanently lost, as they would be sent to an inaccessible address.

Recommendation

Add a zero-check for the new fee recipient.

Status

Resolved

[L-02] PaymaticPayments#createERC20PaymentWithTimeLock - Unbounded Timelock period can delay release of funds indefinitely

Description

The `timelockPeriod` parameter is not validated, potentially allowing extremely long timelock periods that could effectively freeze user funds indefinitely. Users could set or be affected by extremely long timelock periods, preventing access to funds for an unreasonable amount of time.

Recommendation

Implement a reasonable upper bound for the timelock period, such as max of 30 days.

Status

Resolved

QA

[Q-01] Contract - Floating Pragma Usage

Description

pragma solidity ^0.8.0 allows compilation across 0.8.x versions, risking inconsistencies from version differences. Compiling with 0.8.0 vs. 0.8.20 may alter behavior (e.g., gas costs, optimizations).

Recommendation

Lock to a specific version. In our case, lock to 0.8.27

Status

Resolved

Centralisation

The Paymatic project prioritizes security and utility, while progressively aligning with decentralization principles. Recent updates have significantly reduced administrative functionalities, effectively decreasing centralization and advancing the protocol toward a more decentralized architecture.

Conclusion

After Hashlock's analysis, the Paymatic project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au